



ESCOLA SUPERIOR DE TECNOLOGIA E GESTÃO
LICENCIATURA EM ENGENHARIA INFORMÁTICA

RELATÓRIO TÉCNICO

Rui Parreira



Beja, maio de 2026

INSTITUTO POLITÉCNICO DE BEJA
ESCOLA SUPERIOR DE TECNOLOGIA E GESTÃO
LICENCIATURA EM ENGENHARIA INFORMÁTICA

RELATÓRIO TÉCNICO

Rui Parreira 25803

Beja, maio de 2026

Índice

1. Introdução	3
2. Metodologia de Desenvolvimento	4
3. Análise de Requisitos	5
3.1. Recolha de Informação	5
3.2. Requisitos Funcionais	5
3.3. Requisitos Não Funcionais	6
4. Casos de Uso	7
4.1. Diagrama de Casos de Uso	7
4.2. Especificação de Casos de Uso	8
5. Desenho	11
5.1. Diagrama de Classes	11
5.2. Diagramas de Sequência	12
6. Padrões Arquiteturais e de Desenho	15
6.1. Padrão Arquitetural – MVC	15
6.2. Padrões de Desenho	16
7. Inovação	18
8. Gestão do Projeto	19
8.1. Ferramentas de Gestão	19
8.2. Controlo de Versões	19
8.3. Validação	19

Introdução

O projeto U-Bike Portugal, coordenado pelo IMT, I.P., tem como objetivo promover a mobilidade suave e sustentável nas comunidades académicas do ensino superior. No IPBeja, o projeto "A Academia a Pedalar" disponibiliza **200 bicicletas** à comunidade académica.

Objetivos do trabalho:

- Desenvolver uma plataforma para monitorização do uso das bicicletas
- Implementar dois indicadores obrigatórios:
 - **Eficiência energética** (CO₂ evitado)
 - **Hábitos de vida saudáveis** (calorias queimadas)

2. METODOLOGIA DE DESENVOLVIMENTO

Foi adotada a metodologia ágil **Scrum**, organizada em 4 Sprints com duração total de 5 semanas.

Sprint	Duração	Objetivo	Entregáveis
Sprint 0	Semana 1	Configuração do ambiente	Trello, Visual Paradigm
Sprint 1	Semana 2	Análise de requisitos	Diagrama de casos de uso, requisitos
Sprint 2	Semanas 3-4	Desenho UML	Diagramas de classes, sequência, padrões
Sprint 3	Semana 5	Consolidação	Relatório final, validação

Ferramentas utilizadas:

Atividade	Ferramenta
Gestão do projeto	Trello
Diagramas UML	Visual Paradigm
Controlo de versões	GitHub
Documentação	Google Docs

Link do projeto Trello:

<https://trello.com/invite/b/69ee3911878216e2b63bdfd2/ATTI33665bcfc5c9f9047a16bad9836da5dcA8748AAB/projes>

3. ANÁLISE DE REQUISITOS

3.1. Recolha de Informação

A recolha de informação foi realizada através das seguintes fontes:

1. **Análise de sistemas similares:**
 - U-Bike IPLeiria
 - U-Bike IPBeja
2. **Pesquisa documental:**
 - Dados concretos do IPBeja: 200 bicicletas

3.2. Requisitos Funcionais

ID	Requisito	Descrição
RF01	Registar Utilizador	Comunidade académica regista-se com email institucional
RF02	Autenticar	Login com credenciais
RF03	Consultar Bicicletas	Visualizar lista com tipo (elétrica/convencional)
RF04	Requisitar Aluguer	Requisitar bicicleta por um período de tempo
RF05	Devolver Bicicleta	Registar devolução e atualizar estado
RF06	Registar Viagem	Guardar distância, duração, consumo
RF07	Calcular Calorias	Calcular kcal gastas por viagem
RF08	Calcular CO ₂ Evitado	Calcular kg CO ₂ evitados
RF09	Reportar Avaria	Comunicar problema na bicicleta
RF10	Consultar Histórico	Visualizar todas as viagens
RF11	Gerir Frota (Admin)	Adicionar/remover bicicletas

3.3. Requisitos Não Funcionais

ID	Requisito	Descrição
RNF01	Segurança	Autenticação obrigatória
RNF02	Usabilidade	Interface simples e intuitiva
RNF03	Disponibilidade	Sistema disponível 24/7
RNF04	Desempenho	Respostas em < 3 segundos
RNF05	Conformidade	Cumprir RGPD

4. CASOS DE USO

4.1. Diagrama de Casos de Uso

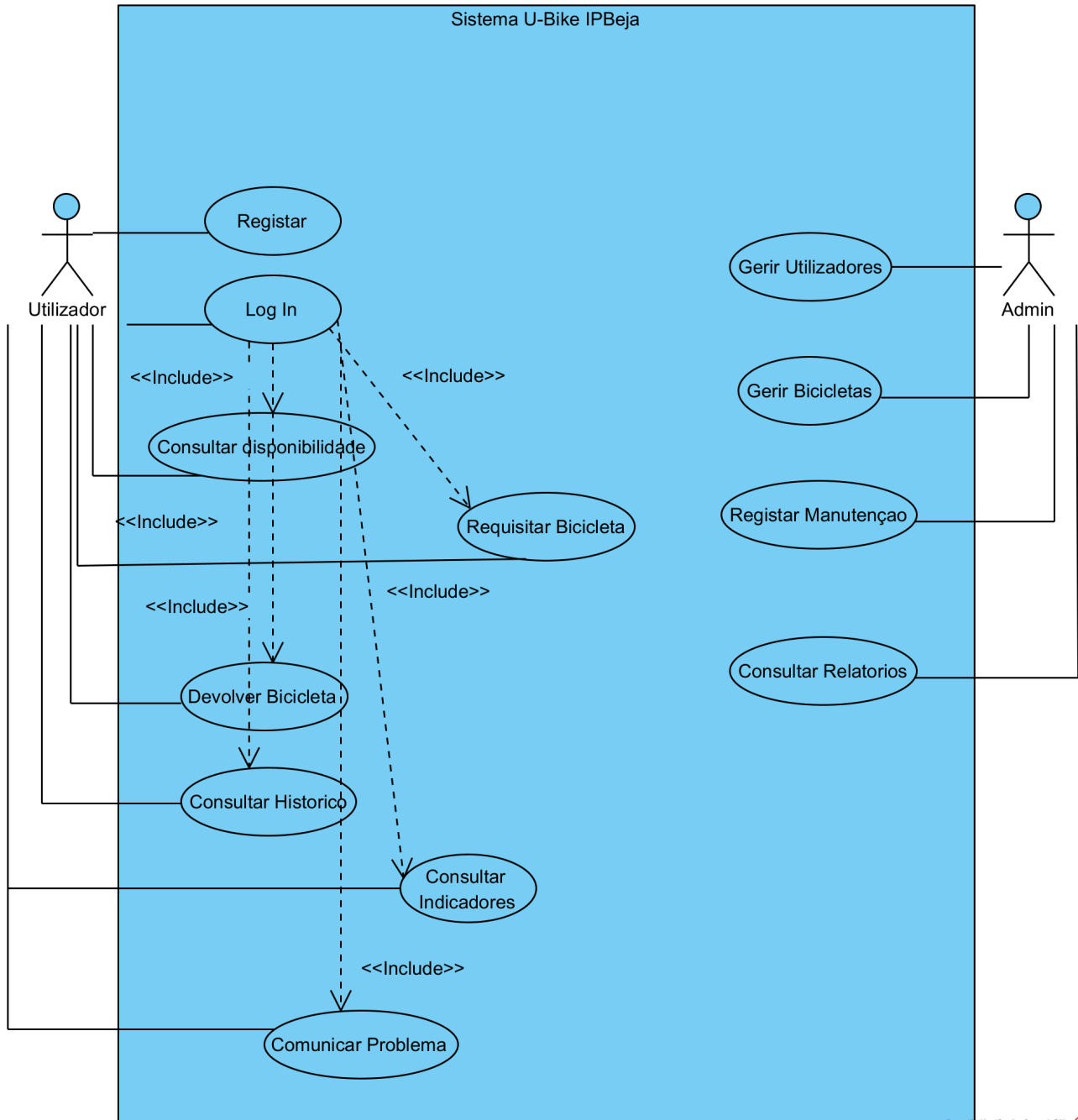


Figura 1 – Diagrama de Casos de Uso do Sistema U-Bike IPBeja

Atores identificados:

- **Utilizador** – Comunidade académica (alunos, docentes, funcionários)
- **Administrador** – Gestor do sistema de bicicletas

Casos de Uso:

- Utilizador: Registrar-se, Fazer login, Consultar bicicletas, Requisitar Aluguer, Devolver bicicleta, Consultar Histórico, Consultar indicadores, Comunicar Problema
- Administrador: Gerir utilizadores, Gerir bicicletas, Registrar manutenções, Consultar relatórios

4.2. Especificação de Casos de Uso

Caso de Uso 1 – Registrar

Descrição O utilizador cria uma conta no sistema fornecendo os seus dados pessoais e institucionais.

Pre-condições: O utilizador não possui uma conta ativa.

Pos-condições: O utilizador fica registado e pode autenticar-se.

Fluxo Principal

- 1 O utilizador insere os dados necessarios
- 2 O sistema valida os dados
- 3 O sistema cria o registo do utilizador com estado "pendente".
- 4 O sistema gera um token de confirmação e envia email.
- 5 O utilizador clica no link de confirmação.
- 6 O sistema ativa a conta (estado "ativo") e exhibe mensagem de sucesso.

Fluxo Alternativo

2a Dados invalidos

O sistema exhibe mensagem de erro e solicita correção. Retorna ao passo 1.

Caso de Uso 2 – Log in

Descrição O utilizador autentica-se no sistema

Pre-condições: O utilizador tem uma conta ativa e confirmada.

Pos-condições: O utilizador obtém um token de sessão e acede às funcionalidades protegidas.

Fluxo Principal

- 1 O utilizador insere os seus dados
- 2 O sistema verifica se as credenciais estão corretas e se a conta está ativa.
- 3 O sistema gera um token de sessão e redireciona para o dashboard.

Fluxo Alternativo

2a Credenciais inválidas

sistema informa “Os dados estão inválidos”. Retorna ao passo 1

2b Conta não ativada

O sistema informa que a conta não foi confirmada e oferece reenvio do email.

Caso de Uso 3 – Consultar disponibilidade

Descrição O utilizador consulta as bicicletas disponíveis numa estação ou para um período específico.

Pre-condições: Utilizador autenticado.

Pos-condições: O sistema exibe a lista de bicicletas disponíveis.

Fluxo Principal

- 1 O utilizador seleciona uma estação ou intervalo de datas.
- 2 O sistema obtém as bicicletas com estado "disponível" na estação/período.
- 3 O sistema apresenta a lista

Fluxo Alternativo

2a – Nenhuma bicicleta disponível

O sistema exibe mensagem “Não há bicicletas disponíveis

Caso de Uso 4 – Requisitar bicicleta

Descrição O utilizador reserva uma bicicleta.

Pre-condições: Utilizador autenticado e existem bicicletas disponíveis.

Pos-condições: A bicicleta fica marcada como "em utilização" e é criado um registo de utilização.

Fluxo Principal

- 1 O utilizador escolhe uma bicicleta da lista de disponíveis.
- 2 O sistema verifica se a bicicleta ainda está disponível.
- 3 O sistema cria uma reserva (estado "reservada") e solicita confirmação de levantamento.
- 4 O utilizador confirma o levantamento
- 5 O sistema altera o estado da bicicleta para "em utilização".
- 6 O sistema cria o registo de utilização com data/hora de início.
- 7 O sistema regista a localização GPS inicial e exibe mensagem de sucesso.

Fluxo Alternativo

4a – Utilizador cancela antes de levantar

O sistema liberta a reserva e mantém a bicicleta como disponível.

5a – Bicicleta já não está disponível (entretanto foi requisitada por outro)

O sistema informa e solicita que o utilizador escolha outra bicicleta. Retorna ao passo 1.

Caso de Uso 5 – Devolver bicicleta

Descrição O utilizador termina a utilização e devolve a bicicleta numa estação.

Pre-condições: O utilizador tem uma bicicleta em seu poder

Pos-condições: A bicicleta fica disponível, a utilização é encerrada e os indicadores são atualizados.

Fluxo Principal

- 1 O utilizador seleciona a utilização ativa e indica a estação de devolução.
- 2 O sistema regista a data/hora de fim e a localização.
- 3 O sistema calcula duração, distância percorrida e calorias gastas.
- 4 O sistema altera o estado da bicicleta para "disponível".
- 5 O sistema atualiza os indicadores de sustentabilidade
- 6 O sistema exibe o resumo da utilização ao utilizador.

Fluxo Alternativo

3a – Falha no cálculo de distância/GPS

O sistema usa a distância estimada com base nas estações de origem/destino.

Caso de Uso 6 – Consultar histórico

Descrição O utilizador visualiza o registo das suas utilizações anteriores.

Pre-condições: Utilizador autenticado.

Pos-condições: O sistema mostra a lista de utilizações

Fluxo Principal

- 1 O utilizador solicita o histórico
- 2 O sistema consulta as utilizações do utilizador.

3 O sistema exibe os dados ordenados por data

Fluxo Alternativo

2a – Nenhuma utilização encontrada

O sistema exibe mensagem “Ainda não realizou nenhuma viagem”.

Caso de Uso 7 – Consultar indicadores

Descrição O utilizador vê os seus indicadores de saúde

Pre-condições: Utilizador autenticado.

Pos-condições: O sistema apresenta os valores acumulados

Fluxo Principal

1 O utilizador seleciona o tipo de indicador e período

2 O sistema obtém os dados de utilização agregados.

3 O sistema calcula os indicadores

4 O sistema exibe os valores

Fluxo Alternativo

2a – Nenhuma utilização encontrada

O sistema exibe mensagem “Ainda não realizou nenhuma viagem”.

Caso de Uso 8 – Comunicar problema

Descrição O utilizador reporta uma avaria ou anomalia numa bicicleta ou estação.

Pre-condições: Utilizador autenticado.

Pos-condições: O problema fica registado para a gestão; o utilizador recebe um número de ticket.

Fluxo Principal

- 1 O utilizador seleciona o tipo de problema
- 2 O utilizador descreve o problema.
- 3 O sistema cria um registo de manutenção com estado "reportado".
- 4 O sistema atribui um identificador único e notifica o gestor.
- 5 O sistema exibe a confirmação ao utilizador.

Fluxo Alternativo

3a – Dados obrigatórios em falta

O sistema informa quais campos são necessários e solicita preenchimento. Retorna ao passo 1.

Caso de Uso 9 – Gerir utilizadores (Administrador)

Descrição O administrador pode listar, bloquear, ativar ou editar dados de utilizadores.

Pre-condições: Administrador autenticado com permissões de gestão.

Pos-condições: As alterações são aplicadas e registadas.

Fluxo Principal

- 1 O administrador acede à lista de utilizadores
- 2 O sistema exibe os utilizadores.
- 3 O administrador seleciona um utilizador e a ação (bloquear, ativar, editar).
- 4 O administrador confirma a ação.
- 5 O sistema executa a ação e regista a alteração

Fluxo Alternativo

3a – Utilizador com utilizações ativas (tentativa de bloquear)

O sistema informa que não é possível bloquear enquanto houver uma bicicleta em seu poder.

Caso de Uso 10 – Gerir bicicletas (Administrador)

Descrição O administrador adiciona, edita, remove bicicletas ou altera o estado

Pre-condições: Administrador autenticado.

Pos-condições: O catálogo de bicicletas é atualizado.

Fluxo Principal

- 1 O administrador seleciona a opção de gestão de bicicletas.
- 2 O sistema apresenta a lista de bicicleta
- 3 O administrador escolhe uma ação: adicionar, editar, remover ou alterar estado.
- 4 O administrador preenche os dados necessários
- 5 O sistema valida os dados e executa a operação.

Fluxo Alternativo

4a – Remoção de bicicleta com utilizações ativas

O sistema impede a remoção e informa que a bicicleta deve primeiro ser devolvida.

Caso de Uso 11 – Registrar manutenção (Administrador)

Descrição O administrador regista uma intervenção de manutenção numa bicicleta.

Pre-condições: Administrador autenticado. A bicicleta existe.

Pos-condições: A bicicleta fica com estado "em manutenção" e é criado um registo de manutenção.

Fluxo Principal

- 1 O administrador seleciona a bicicleta.
- 2 O administrador insere tipo de manutenção, descrição, data prevista de fim.

- 3 O sistema altera o estado da bicicleta para "em manutenção".
- 4 O sistema cria o registo de manutenção
- 5 O sistema notifica os utilizadores com reservas para essa bicicleta

Fluxo Alternativo

6a – Finalizar manutenção

O administrador regista a data de fim real. O sistema altera o estado da bicicleta para "disponível" e fecha o registo.

Caso de Uso 12 – Consultar relatórios (Administrador)

Descrição O administrador gera relatórios de utilização, eficiência energética, manutenção ou utilizadores.

Pre-condições: Administrador autenticado.

Pos-condições: O sistema produz um relatório

Fluxo Principal

- 1 O administrador seleciona o tipo de relatório
- 2 O administrador define os filtros
- 3 O sistema consulta os dados conforme os critérios.
- 4 O sistema gera o relatório

Fluxo Alternativo

3a – Nenhum dado para os critérios selecionados

O sistema informa que não existem registos e sugere alterar os filtros.

5. DESENHO

5.1. Diagrama de Classes

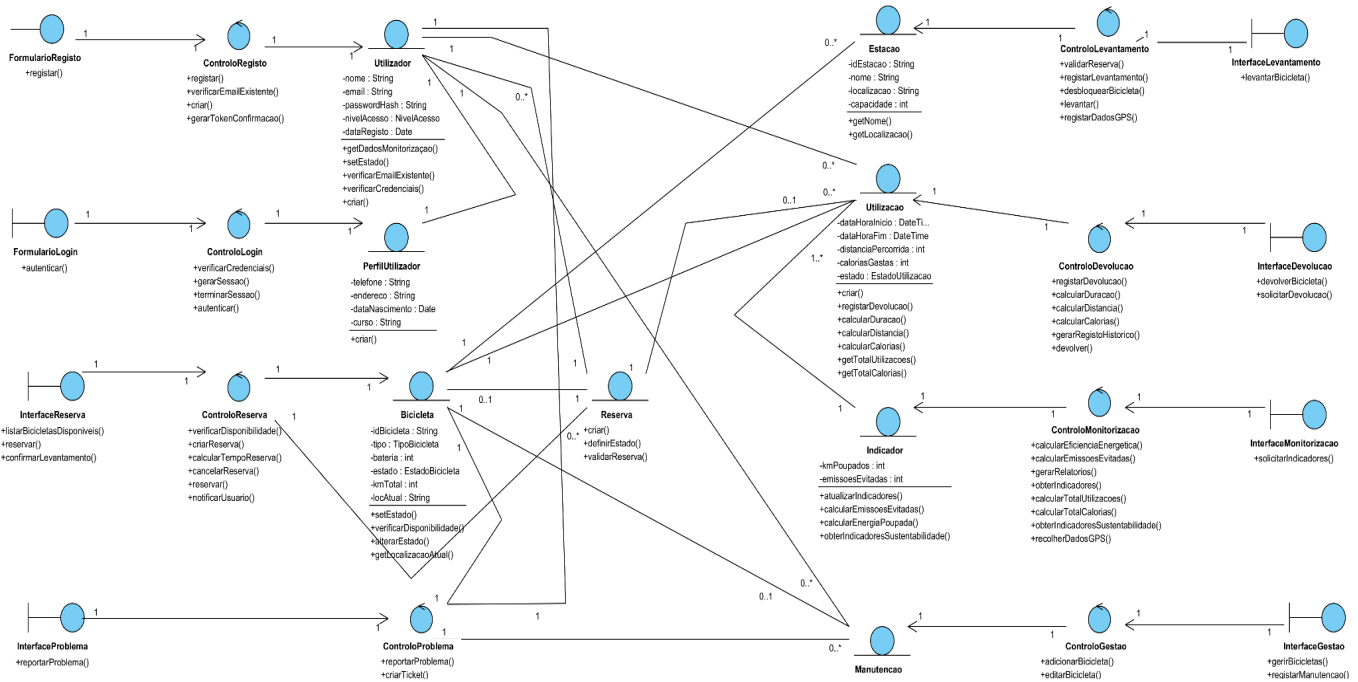


Figura 2 – Diagrama de Classes do Sistema U-Bike IPBeja

O diagrama de classes apresentado modela a estrutura estática do sistema U-Bike IPBeja, seguindo a arquitetura MVC (Model-View-Controller). Foram utilizados os estereótipos <<boundary>>, <<control>> e <<entity>> para separar claramente as responsabilidades.

Classes de fronteira (<<boundary>>)

Representam as interfaces com os atores. Por exemplo:

- FormularioRegisto – recolhe os dados de registo do utilizador.
- InterfaceReserva – permite consultar disponibilidade e reservar bicicletas.
- InterfaceGestao – disponibiliza as funcionalidades de administração.

Classes de controlo (<<control>>)

Contêm a lógica de negócio e coordenam a interação entre a fronteira e as entidades. Cada caso de uso é gerido por um controlador específico:

- ControloRegisto – valida dados, cria utilizador, envia email de confirmação.
- ControloReserva – verifica disponibilidade, cria reservas, gere o estado da bicicleta.
- ControloMonitorizacao – calcula indicadores de saúde (calorias) e ambientais (CO₂ evitado).
- ControloGestao – executa operações de administração (CRUD de bicicletas, utilizadores, manutenções).

Classes de entidade (<<entity>>)

Persistem os dados do domínio. As principais são:

- Utilizador – armazena nome, email, password (hash), estado, nível de acesso.
- Bicicleta – identifica a bicicleta, tipo (elétrica/convencional), estado, nível de bateria.
- Utilizacao – regista cada utilização (data/hora de início e fim, distância, calorias).
- Reserva – guarda as reservas temporárias antes do levantamento.
- Manutencao – regista intervenções de reparação.
- IndicadorSustentabilidade – agrega totais de CO₂ evitado e energia poupada.
- Estacao – representa os pontos de recolha/devolução.

5.2. Diagramas de Sequência

Os diagramas de sequência foram construídos com base nos casos de uso especificados e no diagrama de classes, respeitando a arquitetura MVC. Em todos os diagramas, o ator interage exclusivamente com classes <<boundary>>, estas invocam classes <<control>> e os controladores coordenam as classes <<entity>>. Os nomes das mensagens correspondem a operações definidas nas classes do diagrama de classes.

Objetivo: Modelar o fluxo principal do caso de uso "Registrar".

Participantes: Ator Utilizador; FormularioRegisto (boundary); ControlloRegisto (control); Utilizador, PerfilUtilizador (entity).

Fluxo: O Utilizador envia os dados para o boundary, que chama o controlo. O controlo verifica se o email existe, cria o utilizador e o perfil, gera token de confirmação e retorna sucesso.

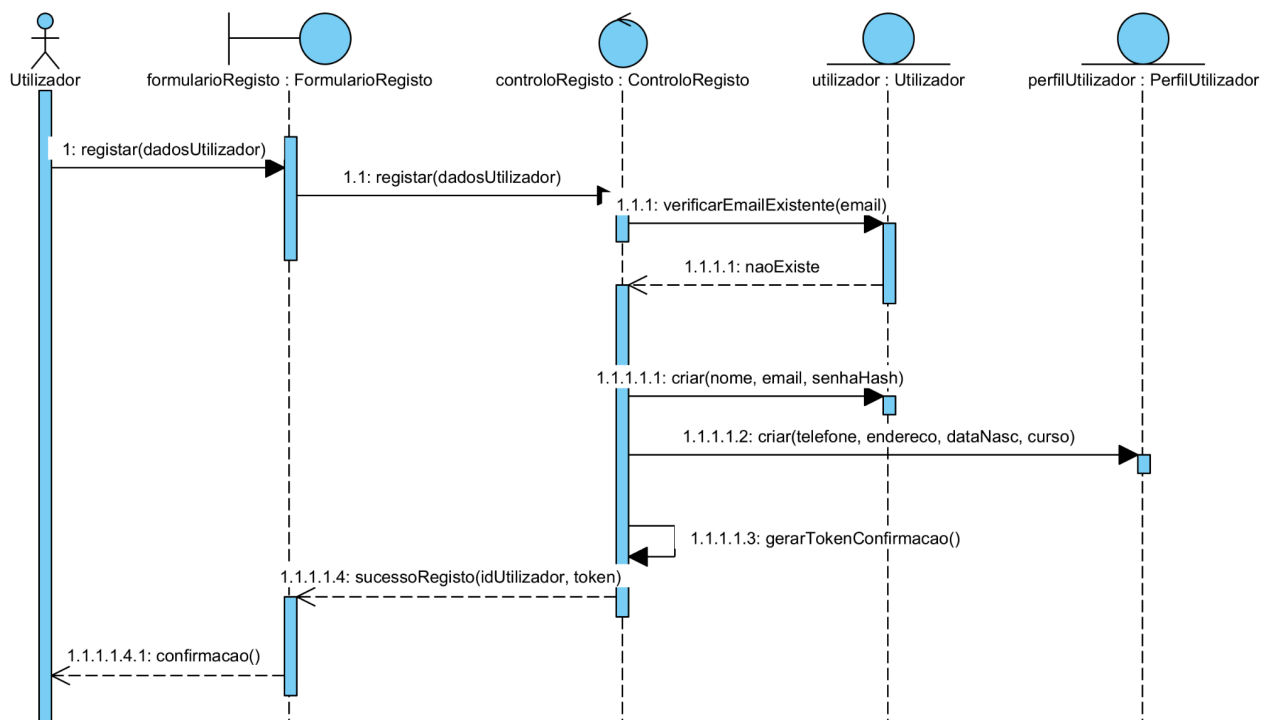


Figura 1 – Diagrama de Sequência: Registrar

Objetivo: Modelar o fluxo principal do caso de uso "Log in".

Participantes: Ator Utilizador; FormularioLogin (boundary);
ControloAutenticacao (control); Utilizador (entity).

Fluxo: O Utilizador insere credenciais no boundary, que invoca o controlo. Este verifica as credenciais na entity e, se válidas, gera uma sessão e redireciona.

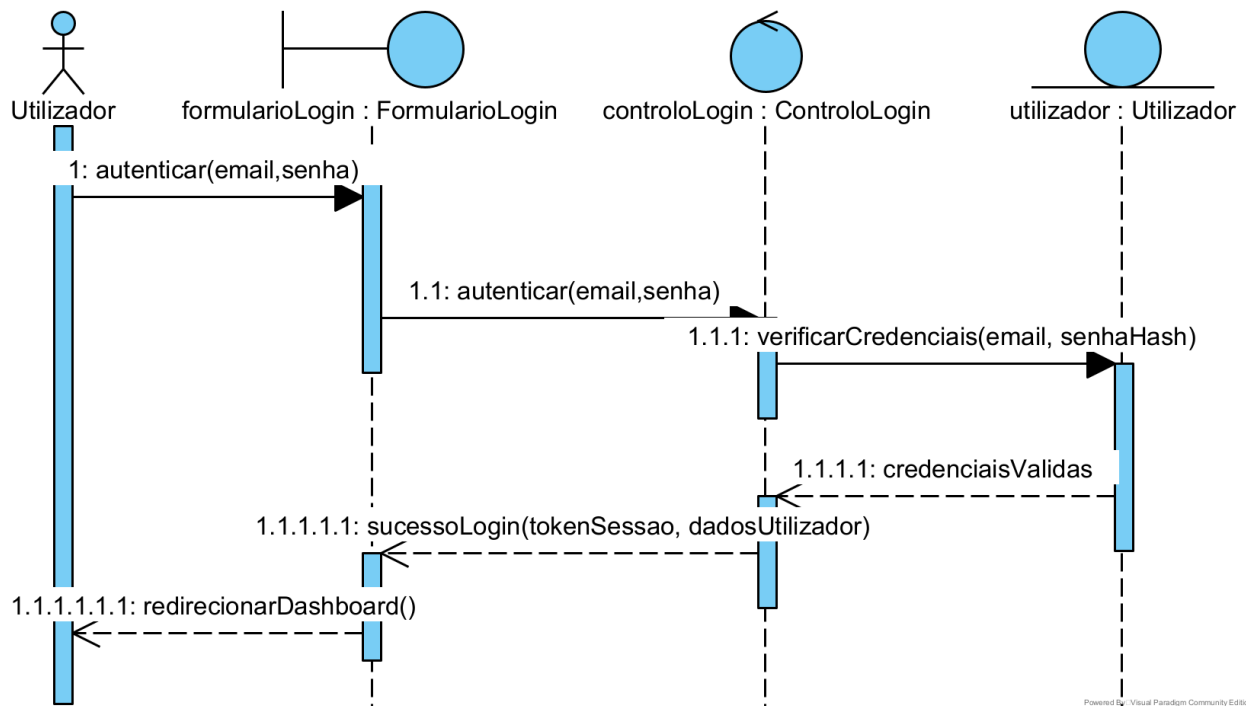


Figura 2 – Diagrama de Sequência: Log In

Objetivo: Modelar a consulta de bicicletas disponíveis.

Participantes: Ator Utilizador; InterfaceReserva (boundary); ControloReserva (control); Bicicleta (entity).

Fluxo: O Utilizador indica estação/período ao boundary, que chama o controlo. O controlo obtém a lista de bicicletas disponíveis junto da entity e devolve ao boundary para exibição.

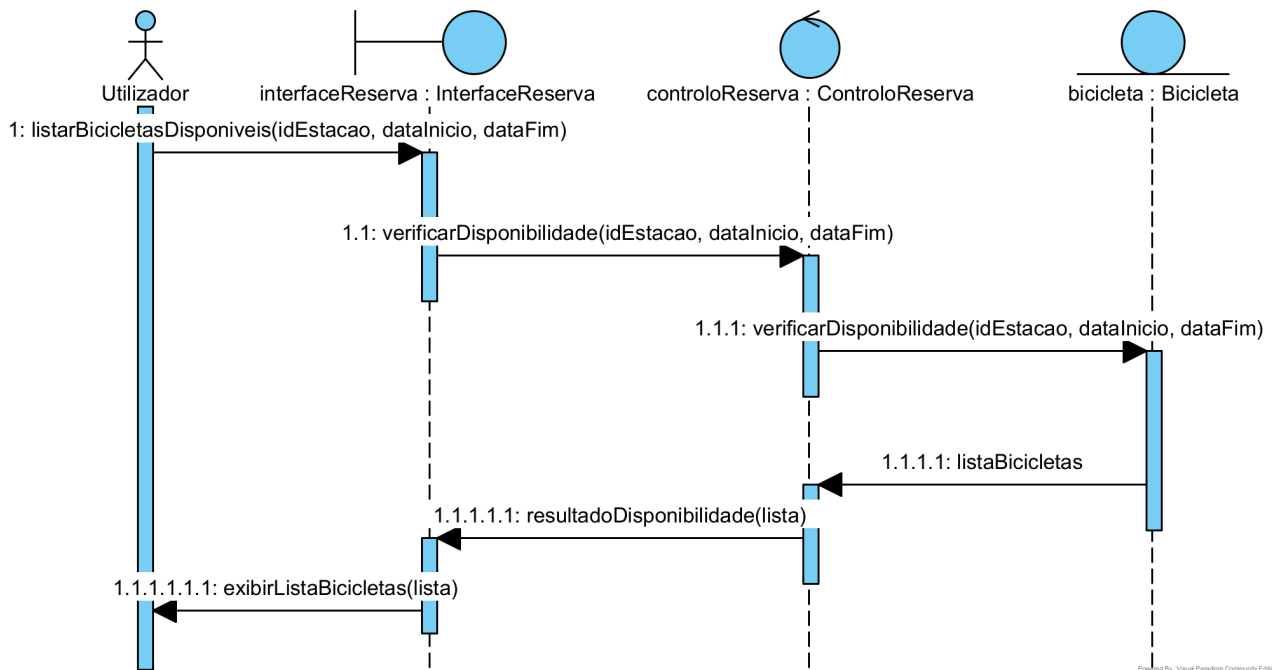


Figura 3 – Diagrama de Sequência: Consultar Disponibilidade

Objetivo: Modelar o processo de reserva e levantamento de uma bicicleta.

Participantes: Ator Utilizador; InterfaceReserva, InterfaceLevantamento (boundary); ControloReserva, ControloLevantamento (control); Bicicleta, Reserva, Utilizacao (entity).

Fluxo: O Utilizador reserva via boundary, o controlo valida disponibilidade e cria reserva. Após confirmação de levantamento, o controlo altera estado da bicicleta, regista utilização e guarda GPS.

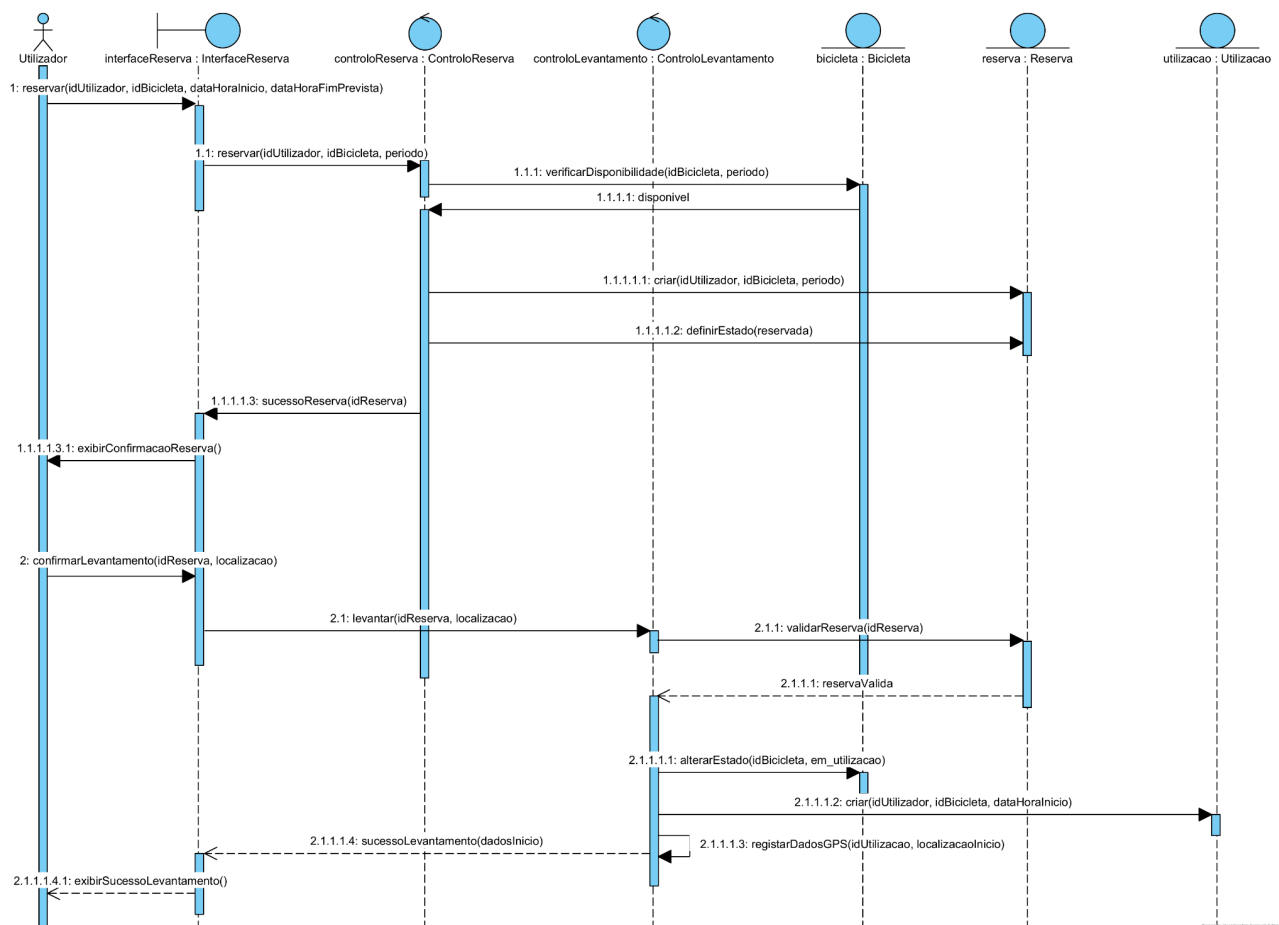


Figura 4 – Diagrama de Sequência: Requisitar Bicicleta

Objetivo: Modelar a devolução da bicicleta e atualização de indicadores.

Participantes: Ator Utilizador; InterfaceDevolucao (boundary); ControloDevolucao (control); Utilizacao, Bicicleta, IndicadorSustentabilidade (entity).

Fluxo: O Utilizador indica devolução ao boundary, que chama o controlo. Este regista o fim da utilização, calcula duração, distância e calorias, altera estado da bicicleta e atualiza os indicadores ambientais.

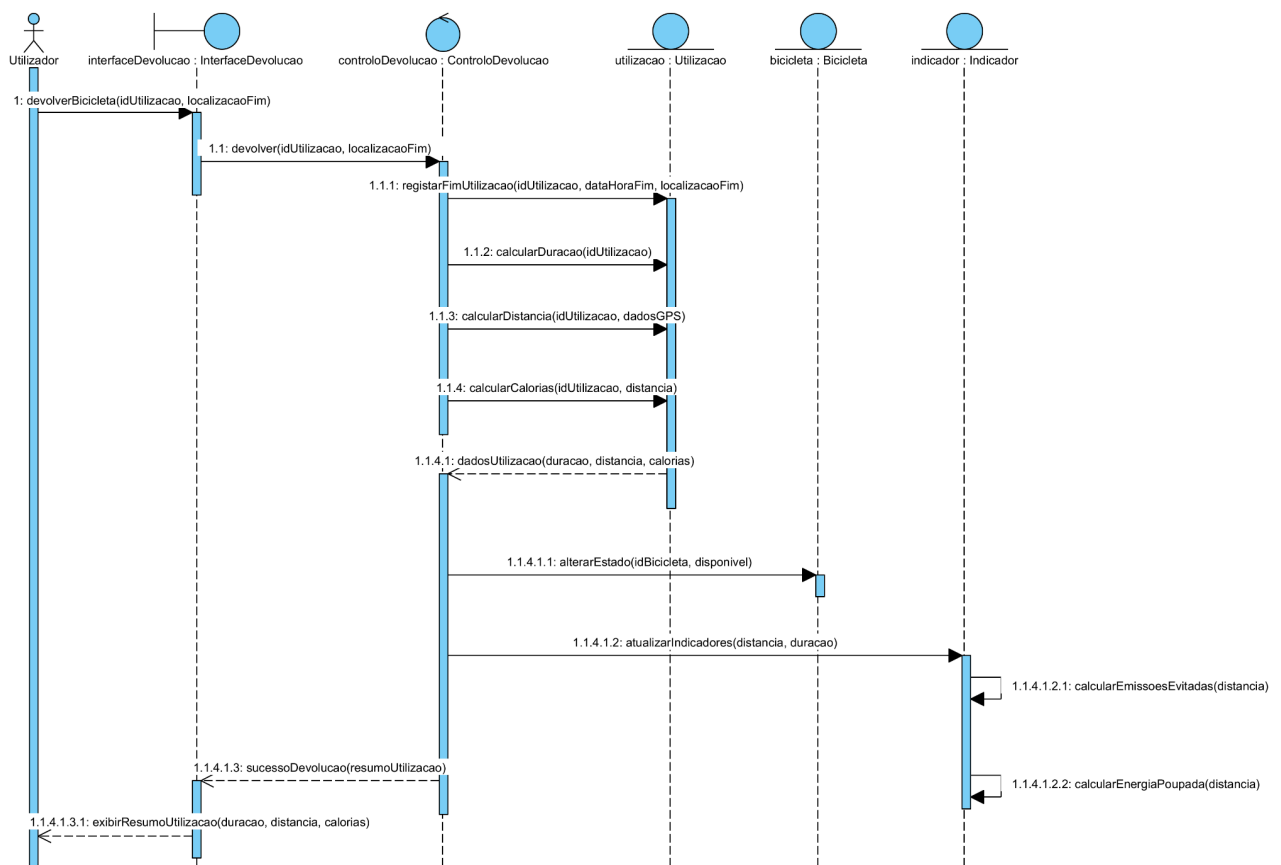


Figura 5 – Diagrama de Sequência: Devolver Bicicleta

Objetivo: Modelar a visualização do histórico de utilizações.

Participantes: Ator Utilizador; InterfaceMonitorizacao (boundary);
ControloMonitorizacao (control); Utilizacao (entity).

Fluxo: O Utilizador solicita o histórico ao boundary, que chama o controlo. Este obtém a lista de utilizações do utilizador junto da entity e envia para exibição.

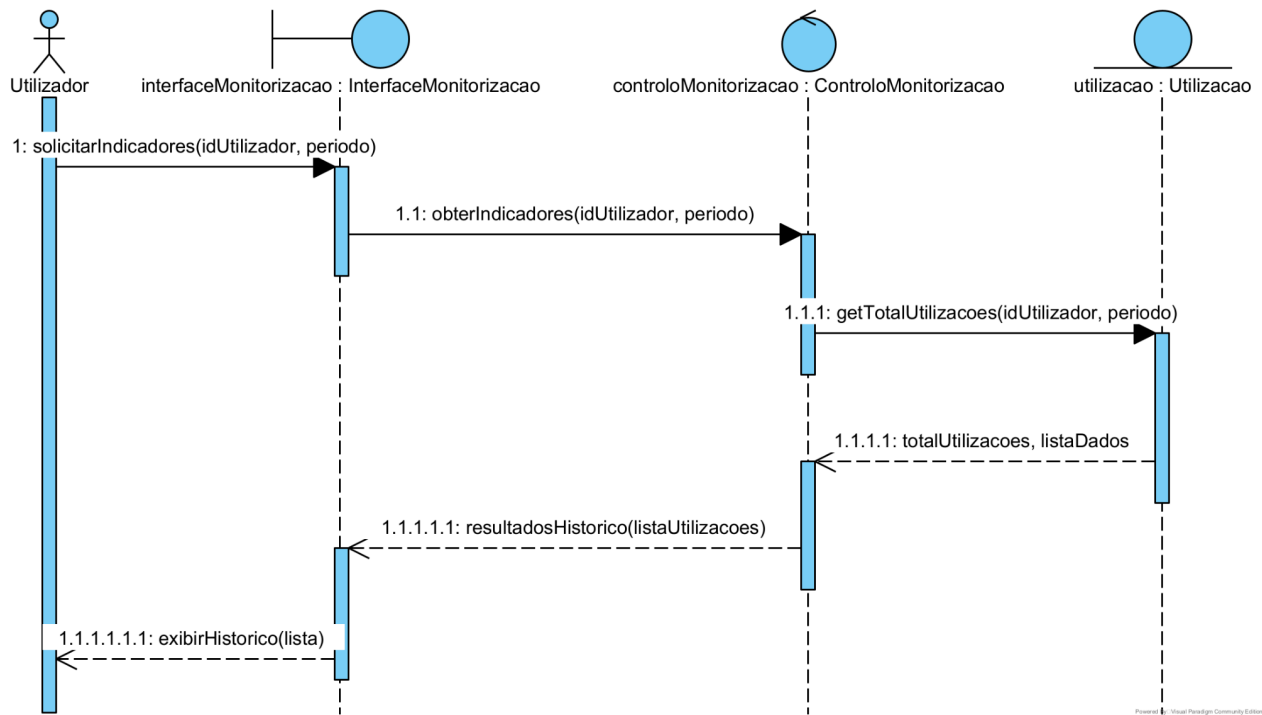


Figura 6 – Diagrama de Sequência: Consultar Histórico

Objetivo: Modelar a consulta de indicadores de saúde e ambientais.

Participantes: Ator Utilizador; InterfaceMonitorizacao (boundary); ControloMonitorizacao (control); Utilizacao, IndicadorSustentabilidade (entity).

Fluxo: O Utilizador seleciona tipo e período no boundary, que chama o controlo. Este calcula totais de calorias e km e obtém os indicadores de sustentabilidade, devolvendo os valores agregados.

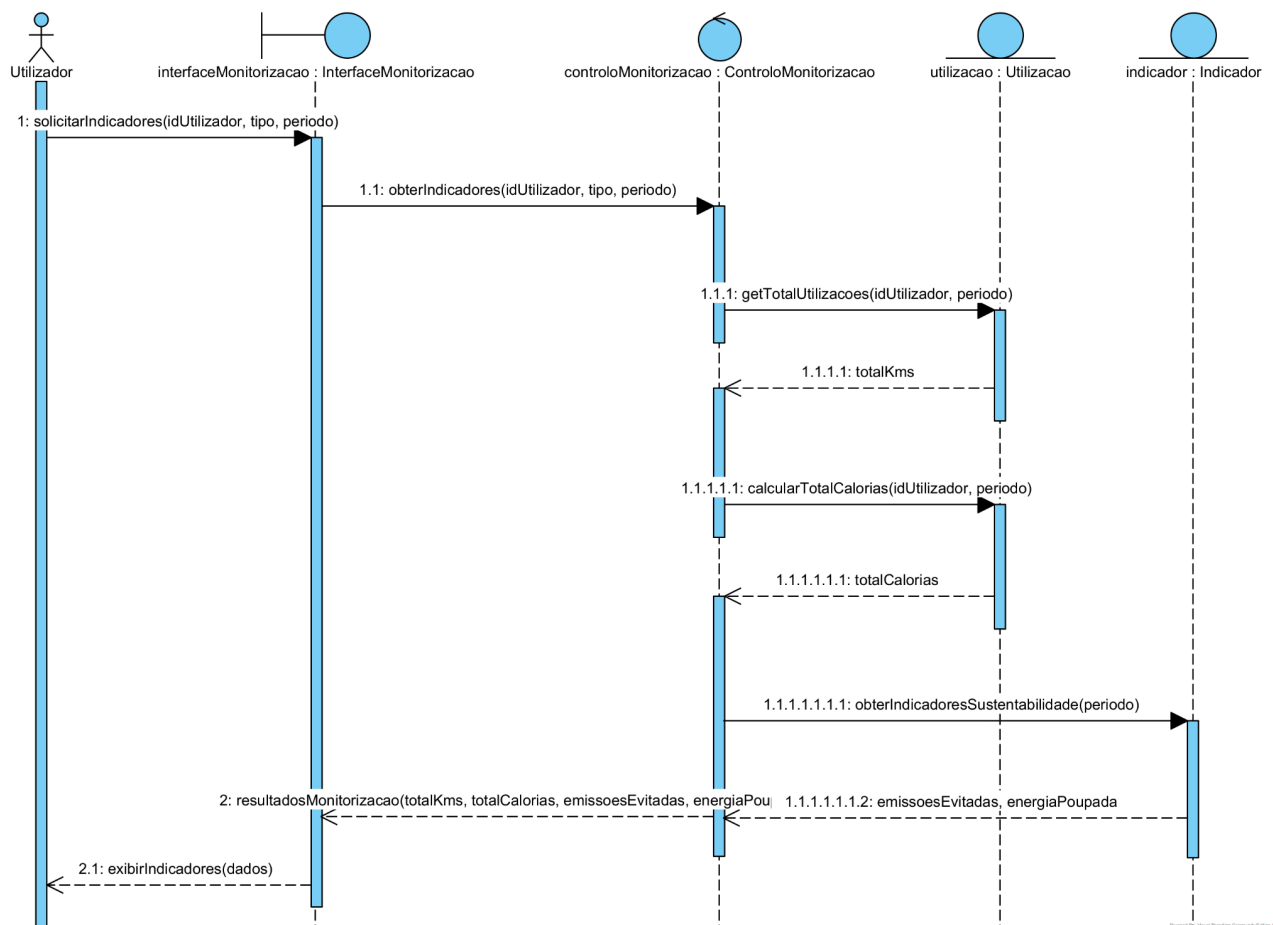


Figura 7 – Diagrama de Sequência: Consultar Indicadores

Objetivo: Modelar o reporte de avaria numa bicicleta ou estação.

Participantes: Ator Utilizador; InterfaceProblema (boundary); ControlloProblema (control); Utilizador, Bicicleta, Manutencao (entity).

Fluxo: O Utilizador descreve o problema no boundary, que chama o controlo. Este valida os dados, cria um registo de manutenção com estado "reportado", notifica o gestor e devolve um ticket ao utilizador.

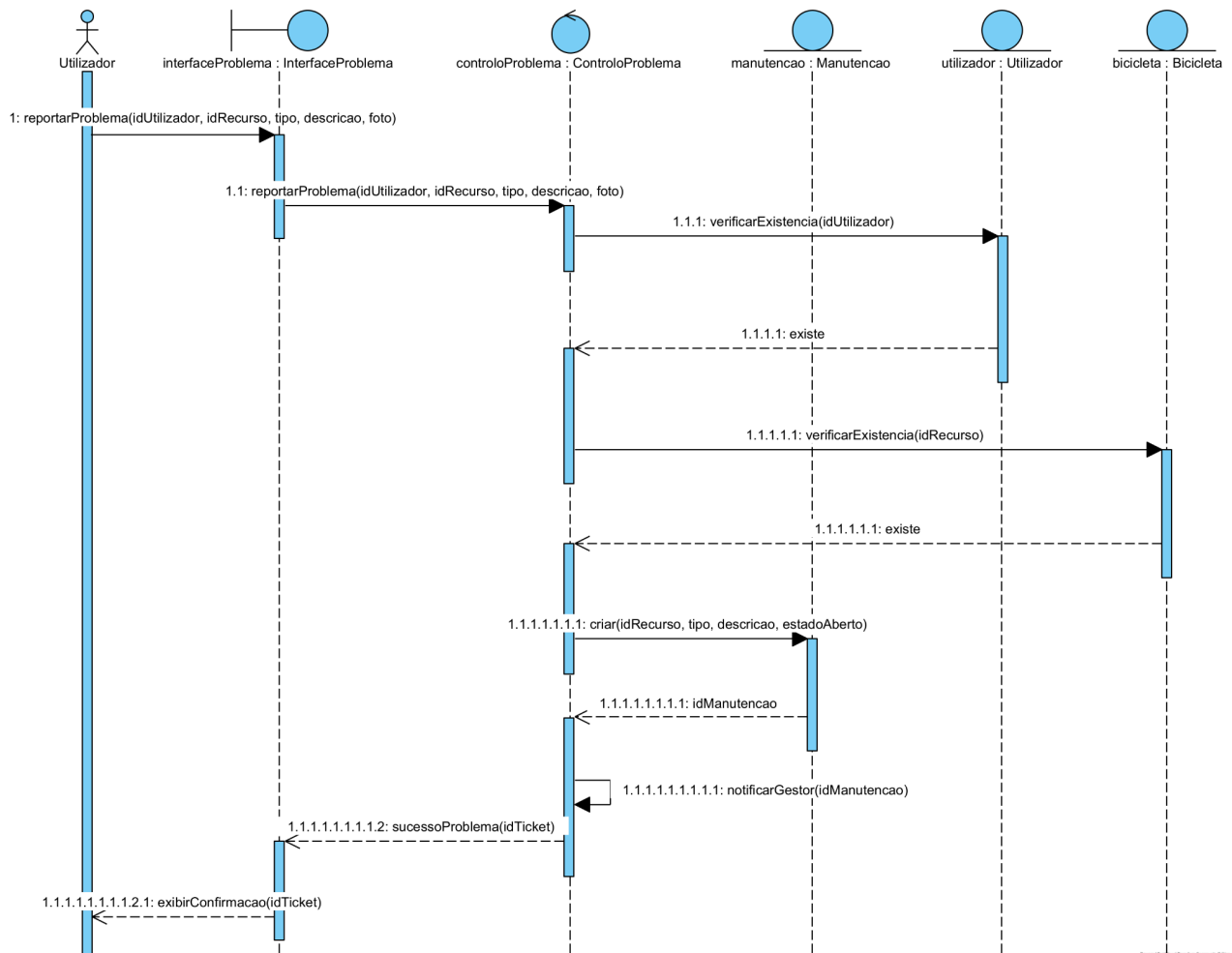


Figura 8 – Diagrama de Sequência: Comunicar Problema

Objetivo: Modelar as ações de gestão de utilizadores (bloquear, ativar, editar).

Participantes: Ator Administrador; InterfaceGestao (boundary); ControloSistema (control); Utilizador (entity).

Fluxo: O Administrador selecciona um utilizador e uma ação no boundary. O controlo executa a operação na entity e regista a alteração.

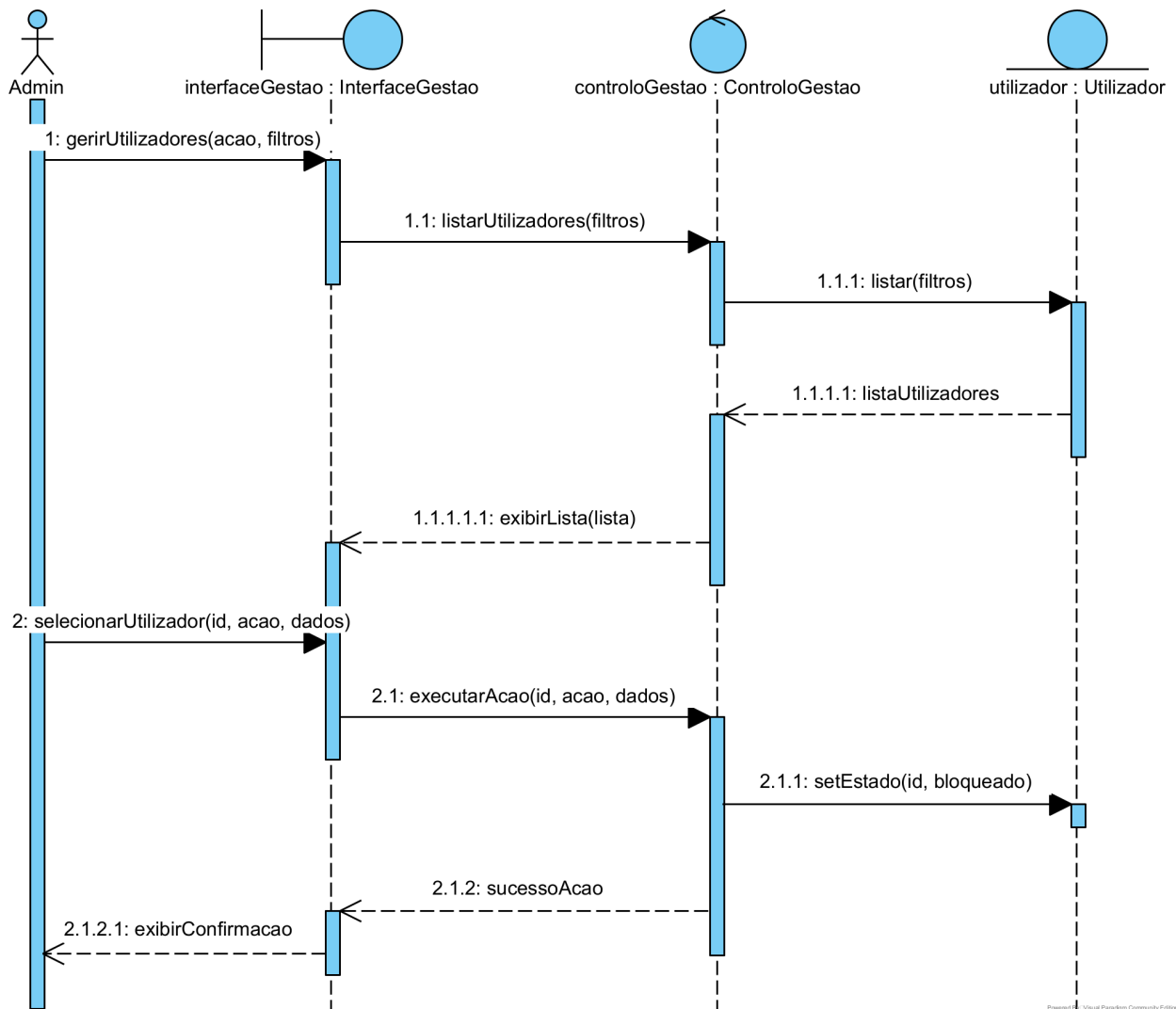


Figura 9 – Diagrama de Sequência: Gerir Utilizadores

Objetivo: Modelar a administração da frota (adicionar, editar, remover, alterar estado).

Participantes: Ator Administrador; InterfaceGestao (boundary); ControloGestao (control); Bicicleta (entity).

Fluxo: O Administrador escolhe a ação e preenche os dados no boundary. O controlo valida, executa a operação na entity e confirma a atualização.

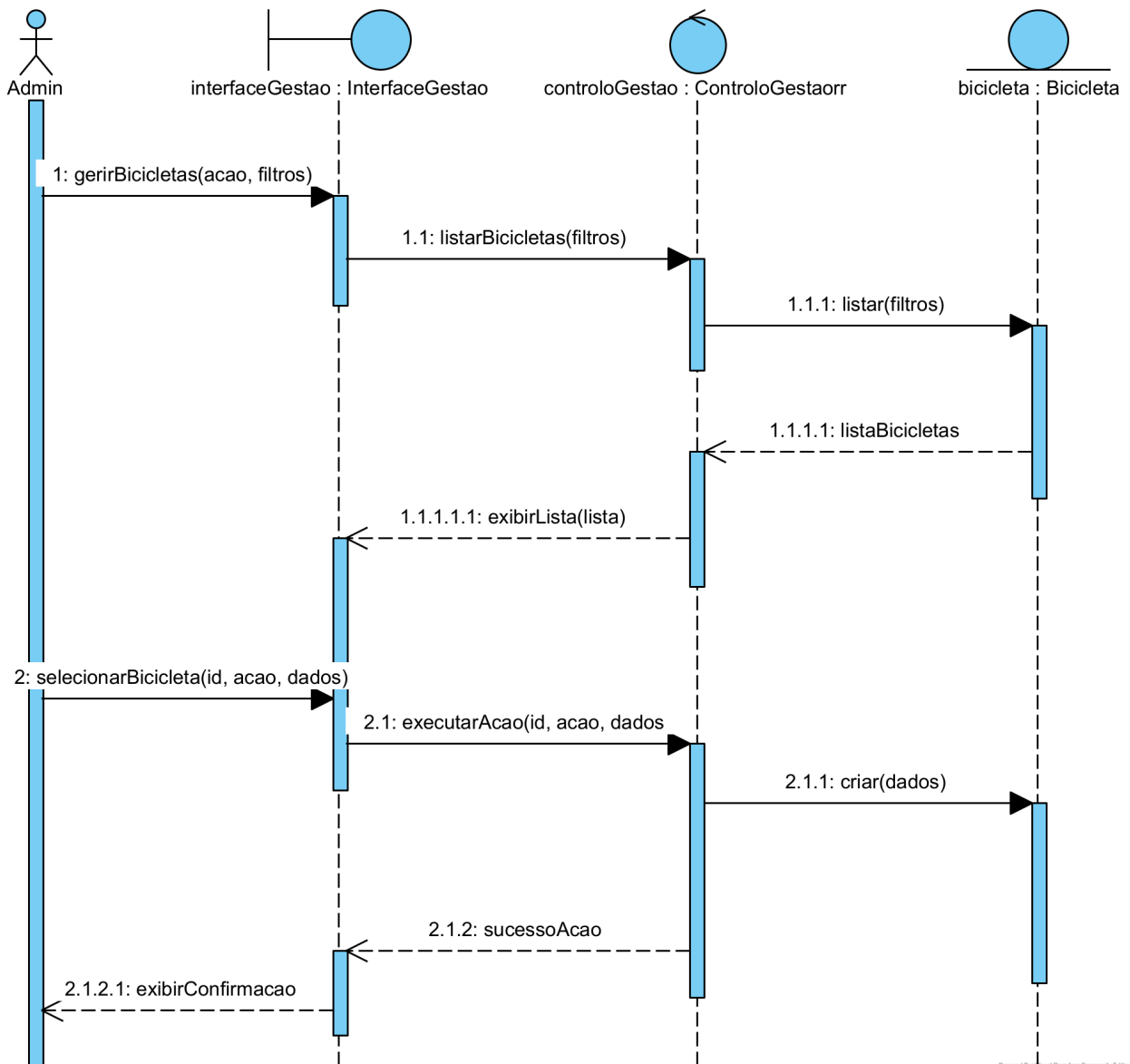


Figura 10 – Diagrama de Sequência: Gerir Bicicletas

Objetivo: Modelar o registo de uma intervenção de manutenção.

Participantes: Ator Administrador; InterfaceGestao (boundary); ControloSistema (control); Bicicleta, Manutencao (entity).

Fluxo: O Administrador indica a bicicleta e os dados da manutenção. O controle altera o estado da bicicleta para "em manutenção", cria o registo de manutenção e notifica os utilizadores com reservas.

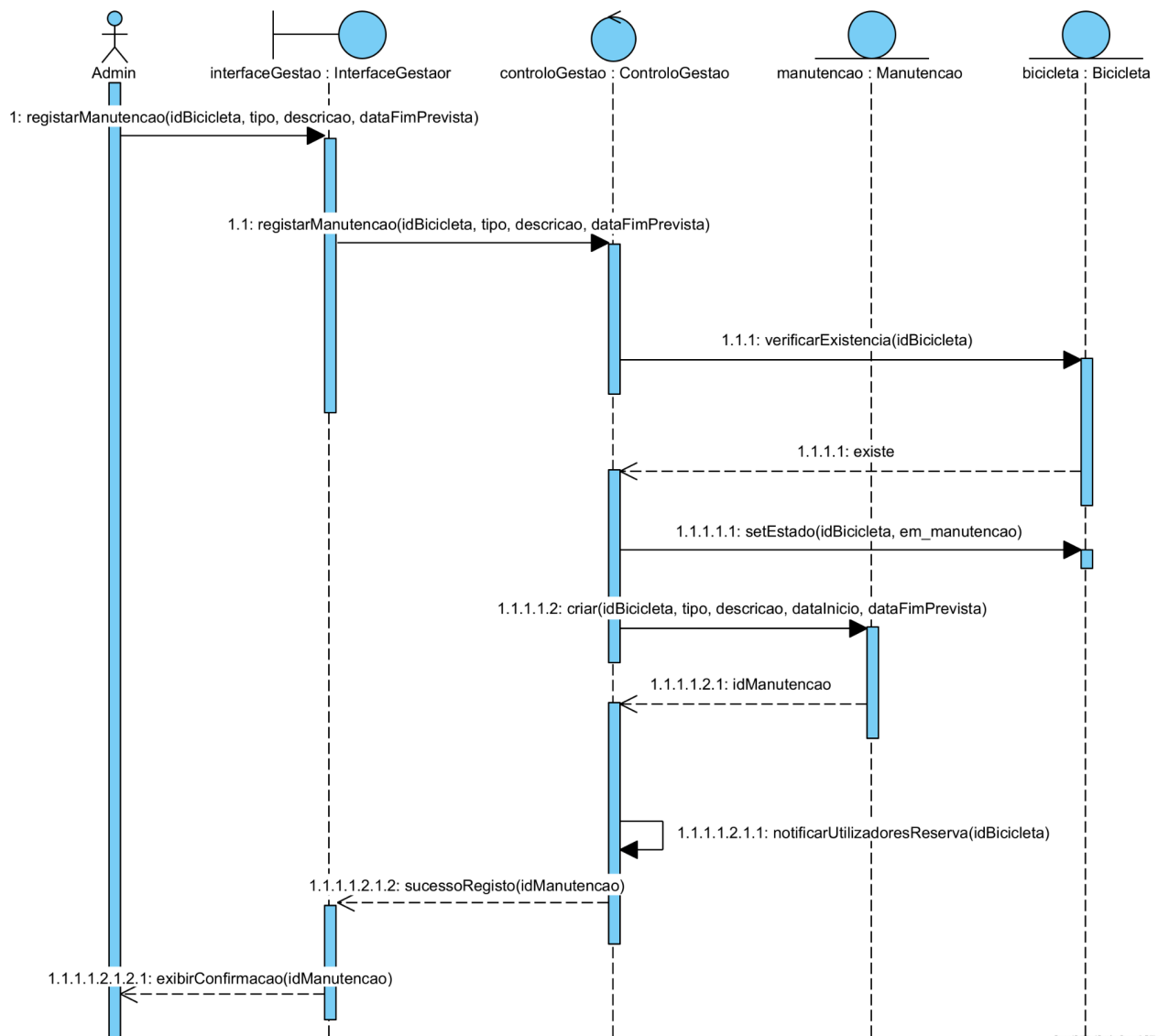


Figura 11 – Diagrama de Sequência:Registrar Manutenção

Objetivo: Modelar a geração de relatórios de utilização, manutenção ou indicadores.

Participantes: Ator Administrador; InterfaceGestao (boundary); ControloGestao (control); Utilizacao, Manutencao, IndicadorSustentabilidade (entity).

Fluxo: O Administrador seleciona o tipo de relatório e filtros. O controlo consulta a entity apropriada, gera o relatório e disponibiliza para download.

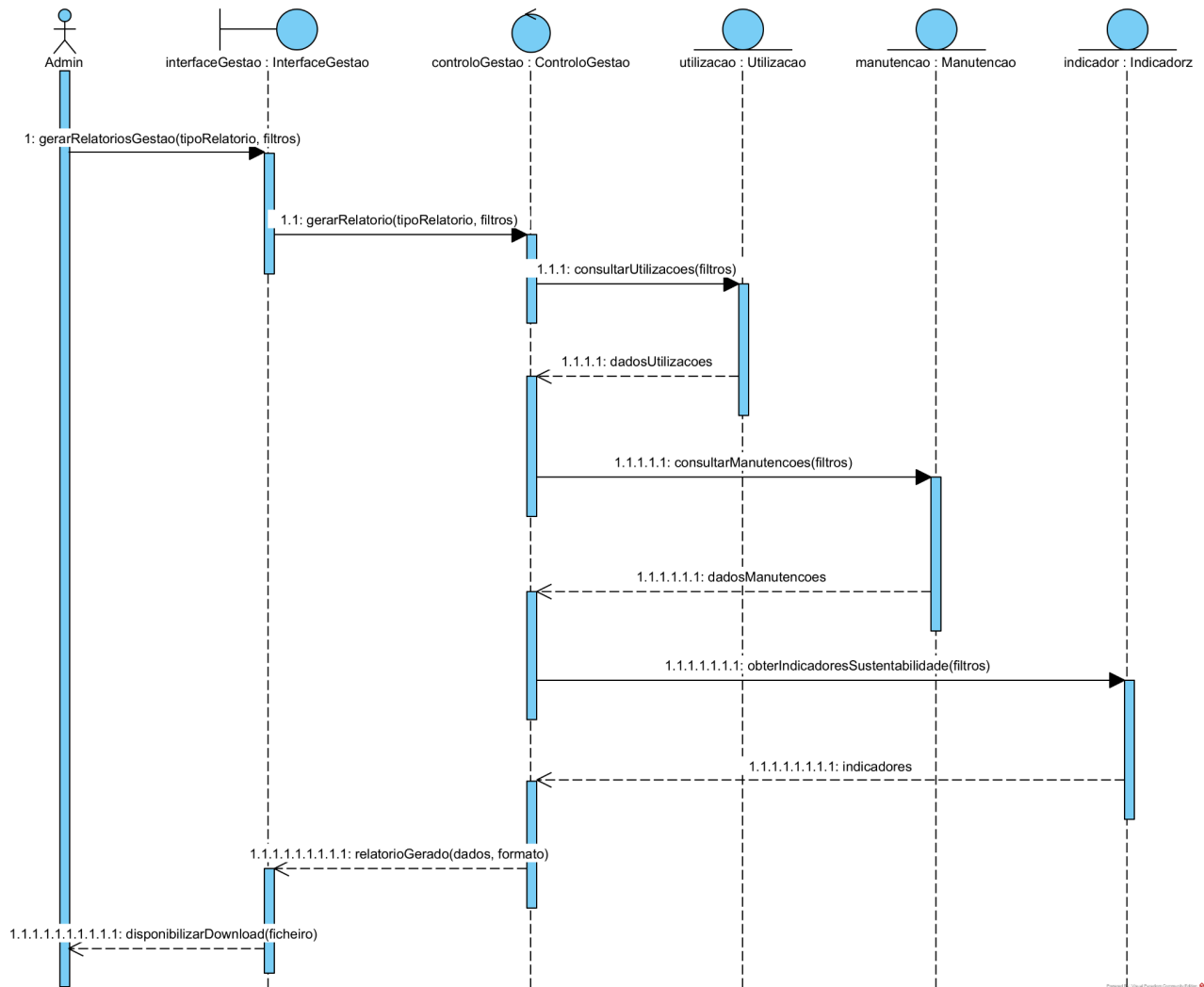


Figura 12 – Diagrama de Sequência: Consultar Relatorios

6. PADRÕES ARQUITETURAIS E DE DESENHO

6.1. Padrão Arquitetural – MVC

A plataforma U-Bike IPBeja foi estruturada segundo o padrão arquitetural **MVC**, que separa a aplicação em três componentes principais:

- **Model (Entity)** – Responsável pelos dados e regras de negócio.
- **View (Boundary)** – Interface com o utilizador.
- **Controller (Control)** – Coordena as ações entre View e Model.

Justificação: O MVC garante a separação de responsabilidades, facilita a manutenção e a evolução do sistema.

6.2. Padrões de Desenho

Padrão Singleton – Gestor da Frota

Problema: O sistema precisa de um ponto central para gerir a frota de bicicletas. Deve existir apenas uma instância deste gestor em toda a aplicação, para evitar inconsistências (ex.: duas partes do sistema a tentar modificar a mesma bicicleta em simultâneo).

Solução: Aplicou-se o padrão **Singleton**, que garante que a classe GestorFrota tenha uma única instância e fornece um ponto global de acesso.

Implementação (pseudocódigo Java):

```
java
public class GestorFrota {
    private static GestorFrota instancia;
    private List<Bicicleta> bicicletas;

    private GestorFrota() {
        bicicletas = new ArrayList<>();
    }

    public static GestorFrota getInstance() {
        if (instancia == null) {
            instancia = new GestorFrota();
        }
        return instancia;
    }

    public void adicionarBicicleta(Bicicleta b) { bicicletas.add(b); }

    public List<Bicicleta> getBicicletasDisponiveis() {
```

```

        return bicicletas.stream()
            .filter(b -> b.getEstado() == EstadoBicicleta.DISPONIVEL)
            .collect(Collectors.toList());
    }
}

```

Padrão Factory – Criação de Bicicletas

Problema: A criação de objetos deve ser centralizada e isolada do resto do código, para facilitar a adição de novos tipos no futuro (ex.: bicicleta de carga).

Solução: O padrão **Factory Method** encapsula a lógica de instanciação.

Implementação (pseudocódigo Java):

```
java
```

```

public abstract class Bicicleta {
    protected String codigo;
    protected EstadoBicicleta estado;
    // construtores, getters e setters
}

```

```

public class BicicletaEletrica extends Bicicleta {
    private int bateria; // percentagem 0-100
    public BicicletaEletrica(String codigo, int bateria) {
        this.codigo = codigo;
        this.bateria = bateria;
        this.estado = EstadoBicicleta.DISPONIVEL;
    }
}

```

```

public class BicicletaConvencional extends Bicicleta {
    public BicicletaConvencional(String codigo) {
        this.codigo = codigo;
        this.estado = EstadoBicicleta.DISPONIVEL;
    }
}

```

```

public class BicicletaFactory {
    public static Bicicleta criar(String tipo, String codigo) {
        if ("Eletrica".equals(tipo)) {
            return new BicicletaEletrica(codigo, 100);
        } else if ("Convencional".equals(tipo)) {
            return new BicicletaConvencional(codigo);
        }
        throw new IllegalArgumentException("Tipo desconhecido");
    }
}

```

```
}  
}
```

7. INOVAÇÃO

Inovação: Gamificação

- Ranking semanal dos utilizadores com mais km percorridos
- Promove hábitos de vida saudáveis

8. GESTÃO DO PROJETO

8.1. Ferramentas de Gestão

Ferramenta	Objetivo
Trello	Gestão do Product Backlog e Sprints
Visual Paradigm	Diagramas UML
GitHub	Controlo de versões

Estrutura do Trello:

- O que fazer
- A fazer
- Rever
- Concluído

8.2. Controlo de Versões

Foi utilizado um repositório no **GitHub**.

- Os diagramas do Visual Paradigm foram exportados para o repositório
- O relatório final é versionado no mesmo repositório

Link do repositório: <https://github.com/RRBP04/ProjES>

9. CONCLUSÃO

O presente trabalho permitiu desenvolver a plataforma informática para o projeto **U-Bike IPBeja**, abrangendo todas as fases do ciclo de desenvolvimento de software: análise, desenho e gestão.